Craig Dilliplane

craig.t.dilliplane@gmail.com | +1 (609) 287-5572 | New York, NY

Statement on LLM-Assisted Software Development

The rule I follow is plain: I rely only on code I can read, explain, test, and defend in review. I use LLM-assisted development tools, but I do not treat them as authorities. They are useful collaborators as they can suggest directions, surface hidden assumptions, and help me get unstuck, but they can also be wrong with complete conviction.

In practice, I use LLMs for bounded or repetitious tasks such as interpreting uncommon error messages, drafting candidate API calls that I then verify, writing small helper functions, proposing test cases, tightening documentation language, or comparing implementation strategies. They are most useful when I already understand the goal and need a sharper path to it. They are misused when the task hinges on version-specific APIs, scientific assumptions, numerical stability, licensing, or project conventions. That is where plausible scientific output becomes unfounded.

Because of this, my verification process is deliberately skeptical. Since I usually use LLMs for small, targeted outputs, reading generated code line by line is feasible and comprehensive; it tells me what the model actually produced and lets me isolate ambiguous behavior into a minimal reproducible test case. When an API is involved, I check it against the installed library version, official documentation, and developer fora for deprecation notes and recommended usage. Then I test against *expected* behavior, not merely successful execution. For numerical or scientific code, that means checking limiting cases, scaling behavior, stability, conservation or physical plausibility, agreement with known analytic or published examples, and sensitivity to small parameter changes.

Testing and review are where LLM-assisted code either earns its keep or gets discarded. In an open-source setting, I treat keeping AI-assisted changes small, disclosing AI assistance, and only submitting code I could explain to a maintainer as requirements. The point is not to hide the tool or deny its efficiency gain, but to keep the resulting work reviewable, attributable, documented, and shamelessly correct.

A concrete example comes from my differentiable APIC-DFSPH fluid-simulation prototype in Taichi, a Python-embedded language for parallel computation. I wanted to separate differentiable simulation logic from diagnostic and solver operations so that non-differentiable code could not corrupt the gradient path. I used an LLM to explore ways to stop gradients or exclude some operations from automatic differentiation.

The tool generated suggestions that sounded reasonable but did not match how Taichi actually worked in my implementation. It proposed patterns involving `ti.ad.stop_gradient`, `ti.ad.no_grad`, and `grad_replaced`. The larger problem was that the suggestions treated Taichi autodiff as if it behaved like a conventional Python autodiff library, where gradients can be stopped locally around arbitrary operations. In my implementation, the relevant code was inside Taichi kernels and recorded through Taichi's autodiff tape. The generated fixes were therefore either unavailable in my version, applied at the wrong level of abstraction, or incompatible with kernel restrictions.

I caught this by compiling and running the generated changes, then tracing the compiler and runtime errors back to smaller examples. Once I checked the installed API behavior, it became clear that the generated solution was not something I could patch in place. I discarded it and restructured the code around supported kernel boundaries instead. In practice, that meant separating the differentiable update path, solver diagnostics, and visualization logic more explicitly, rather than trying to hide unsupported operations behind a generic stop-gradient pattern. The LLM had produced code that looked like a familiar autodiff solution, but the compiler, the library version, and the numerical method disagreed.

Outside of this, there are places where I would not use LLM output as more than a prompt for my own work. I would not trust it to produce scientific claims without independent verification, derive domain-specific formulas without checking the derivation myself, handle private data or credentials, or generate large changes that make authorship and review harder to trace. In scientific and open-source software, the responsibility stays with its contributor, not with its tools.